EXP 7

2. Error Handling and Unit Testing

Write a Go program that:

(1) handles at least three distinct error scenarios using the error type (e.g., division by zero, negative square root, empty input) with descriptive messages using fmt.Errorf; and



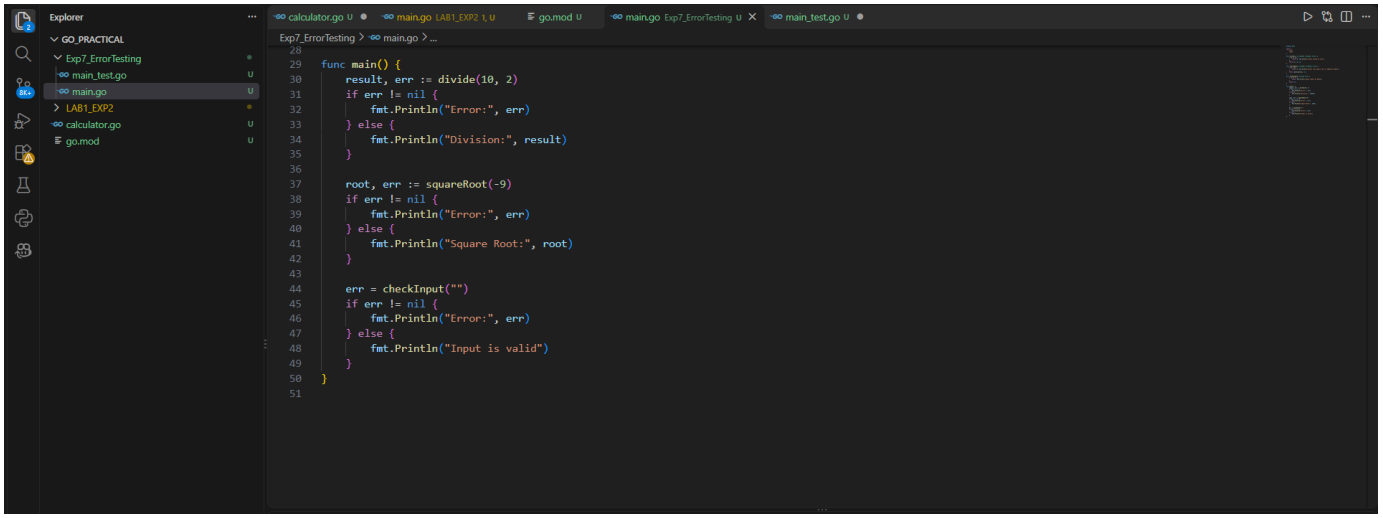
```go
package main

import (
    "fmt"
    "math"
)

func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("cannot divide by zero")
    }
    return a / b, nil
}

func squareRoot(n float64) (float64, error) {
    if n < 0 {
        return 0, fmt.Errorf("cannot find square root of negative number")
    }
    return math.Sqrt(n), nil
}

func checkInput(s string) error {
    if s == "" {
        return fmt.Errorf("input cannot be empty")
    }
    return nil
}
```



```go
func main() {
    result, err := divide(10, 2)
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Division:", result)
    }

    root, err := squareRoot(-9)
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Square Root:", root)
    }

    err = checkInput("")
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Input is valid")
    }
}
```



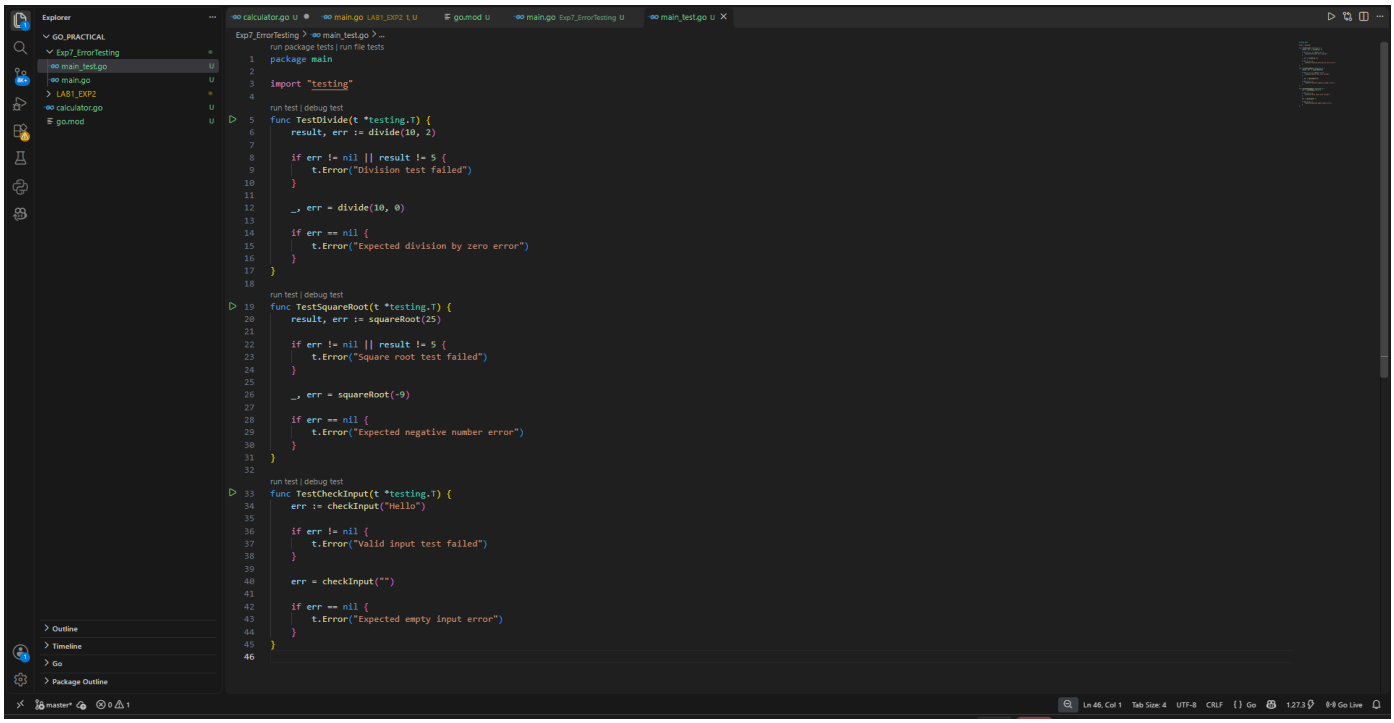
```
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL>go run "c:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting\main.go"
Division: 5
Error: cannot find square root of negative number
Error: input cannot be empty

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL>
```

(2) includes a _test.go file with at least three test functions using the testing package, covering valid inputs and each error case.

- Run go test -v and capture the result.
- Push all source and test files to your GitHub repository. Upload screenshots of go test -v output.



```
package main

import "testing"

func TestDivide(t *testing.T) {
    result, err := divide(10, 2)

    if err != nil || result != 5 {
        t.Error("Division test failed")
    }

    _, err = divide(10, 0)

    if err == nil {
        t.Error("Expected division by zero error")
    }
}

func TestSquareRoot(t *testing.T) {
    result, err := squareRoot(25)

    if err != nil || result != 5 {
        t.Error("Square root test failed")
    }

    _, err = squareRoot(-9)

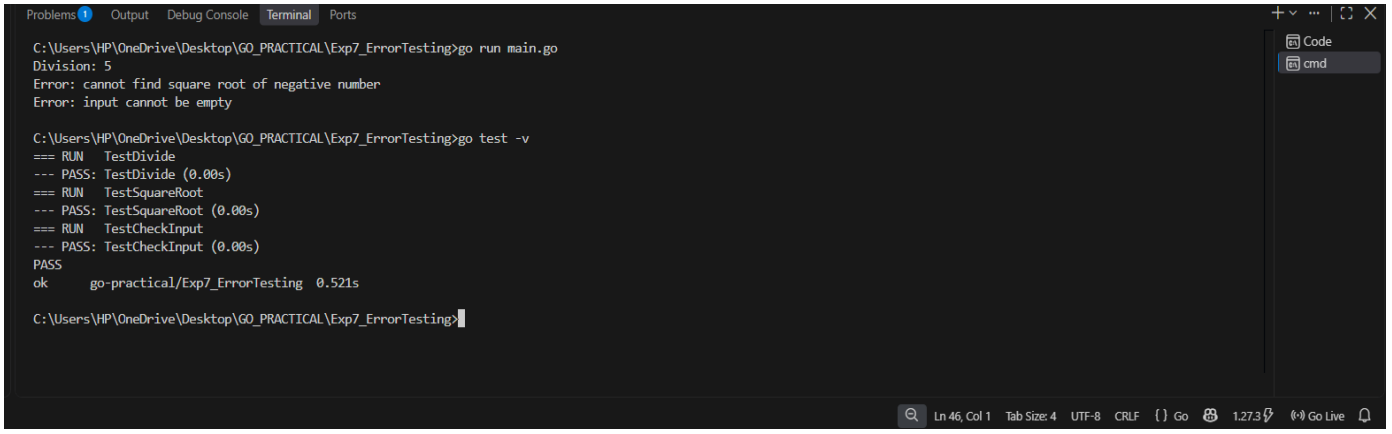
    if err == nil {
        t.Error("Expected negative number error")
    }
}

func TestCheckInput(t *testing.T) {
    err := checkInput("Hello")

    if err != nil {
        t.Error("Valid input test failed")
    }

    err = checkInput("")

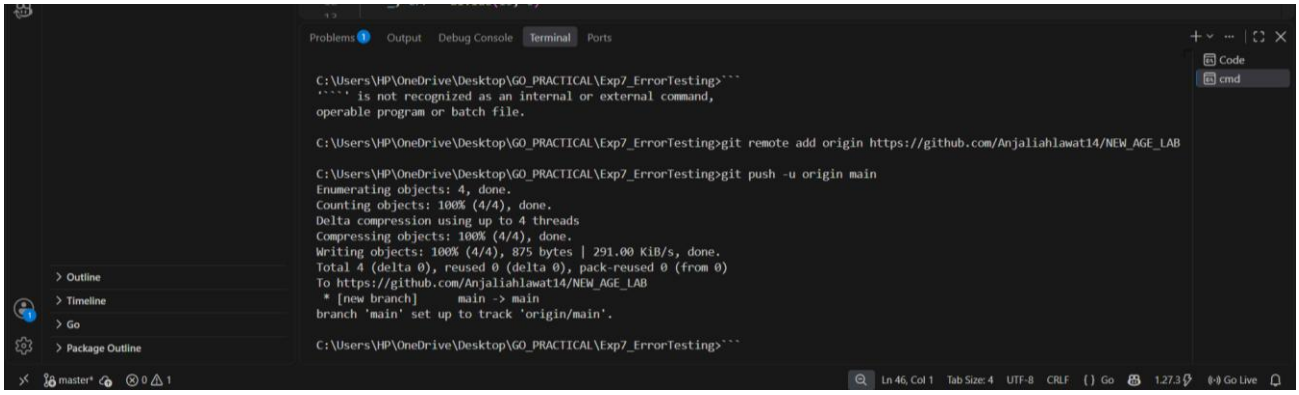
    if err == nil {
        t.Error("Expected empty input error")
    }
}
```



```
C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>go run main.go
Division: 5
Error: cannot find square root of negative number
Error: input cannot be empty

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>go test -v
=== RUN   TestDivide
--- PASS: TestDivide (0.00s)
=== RUN   TestSquareRoot
--- PASS: TestSquareRoot (0.00s)
=== RUN   TestCheckInput
--- PASS: TestCheckInput (0.00s)
PASS
ok      go-practical/Exp7_ErrorTesting  0.521s

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>
```



```
C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>```
'````' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>git remote add origin https://github.com/Anjaliahlawat14/NEW_AGE_LAB

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 875 bytes | 291.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Anjaliahlawat14/NEW_AGE_LAB
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\HP\OneDrive\Desktop\GO_PRACTICAL\Exp7_ErrorTesting>```
```